**EEN-360: Embedded Systems**

# AIR QUALITY MONITORING SYSTEM USING FPGA SPARTAN-3E

BHASKAR GARG (18115027)

PRIYANSHU SINGH SOMVANSHI (18410018)

# Index

# 1. Abstract

With ever-increasing technologies, several new problems are emerging in cities. Air pollution is the most boldly visible, is equally dangerous for the citizens. Still, the exposure of air pollution on cities is poorly understood because of the sparse conventional measurement stations. Sparse because of the high expenses of conventional air quality monitors such as TEOMs, E-BAMs etc. For more relevant air monitoring, it is necessary to have a vast network of sensors spread all across the city. Current researches in the development of low-cost micro-scaled sensing technologies aim to achieve this. These modern aged microsensors have been proved to be highly cost-effective, but the question which arises is of their reliability in terms of accuracy in humid conditions.

The report will focus on devising a new and under research method of PM2.5 estimation using FPGA and frequency calculation. Various algorithms for frequency measurement are deeply discussed. Apart from measurement, taking the desired inputs using I2C communication in FPGA are also explained well. Display of output values using onboard FPGA LCD or bluetooth communication will also be targeted.

# 2. Introduction

Particulate matter count is one of the prime factors in determining the quality of air. The health risk of PM is related to exposure level, leading the EPA and WHO to set air quality standards and guidelines for bothPM2.5 and PM10. The 24-hour standards are 25 g/m3 (WHO) and 35 g/m3 (EPA, U.S) for PM2.5;and 50 g/m3 (WHO) and 150 g/m3 (EPA) for PM10. Studies also show that PM is a common trigger for several chronic lung diseases, such as asthma and COPD . Since these diseases can neither be cured or prevented, lifelong self-management should be implemented for better health outcomes. The first step of prevention comes from accurate detection and awareness regarding the precise pm2.5concentration around us.

Conventionally pm2.5 monitoring stations use TEOM (Tapered Element Oscillating Microbalances) and E-BAMs (Environmental Beta Attenuation Monitors). These sensing systems are costly and costly in Crores for their establishment by the government. The expenses make them very sparse all along with the city. Hence they fail to achieve the primary purpose of measurement. For better understanding, we can take an example of Delhi. Delhi is a region of high importance. Such prominent places also fail to deliver a good network of sensors. There are only 30 air quality monitoring stations spread all across massive 1500 square kilometres of

Delhi. The scenario implies that there is a possibility of having no such station in 10 km radius of my home.

For a better understanding of pm2.5 pollutants, we need a more dense network of pm sensors that are also accurate and affordable. Currently available low-cost pm sensors are not reliav]ble with their accuracy. Ongoing research in the matter tries to give more precise results for low-cost pm sensors. Most of these sensors fail when the humidity is high. Various techniques are under research so that we can have a low-cost sensors network for better air monitoring. Our team has developed a low-cost sensor based on an entirely new principle that assures better results for pm2.5 monitoring.

# 3. Conventional Techniques

Conventionally a wide variety of sensors have been used for pm2.5 monitoring in ambient air. The basic principles used so far can be divided into four major categories as:

1. Gravimetric analysis
2. Optical analysis
3. Beta Attenuation Monitoring
4. Microbalance Methods

Out of these 4 Optical analyses are used in sensors of low cost and carry many sources of inaccuracy; hence these are not reliable to be deployed on large air quality monitoring stations. The trusted government based stations use sensors based on principles of Beta Attenuation Monitoring and Microbalance Methods. These are accurate and very reliable. But they are much more expensive as compared to the low cost sensors. Only one sensor using these principles costs around $20,000. There are separate expenses to deploy them, along with all their connections and software. So, these many expenses make it impossible for the government to deploy such sensors on a large scale and create a robust network of such sensors for effective air quality monitoring.

Follow some more detail about working on each sensor.

## 3.1 TEOM(Tapered Element Oscillating Microbalances)

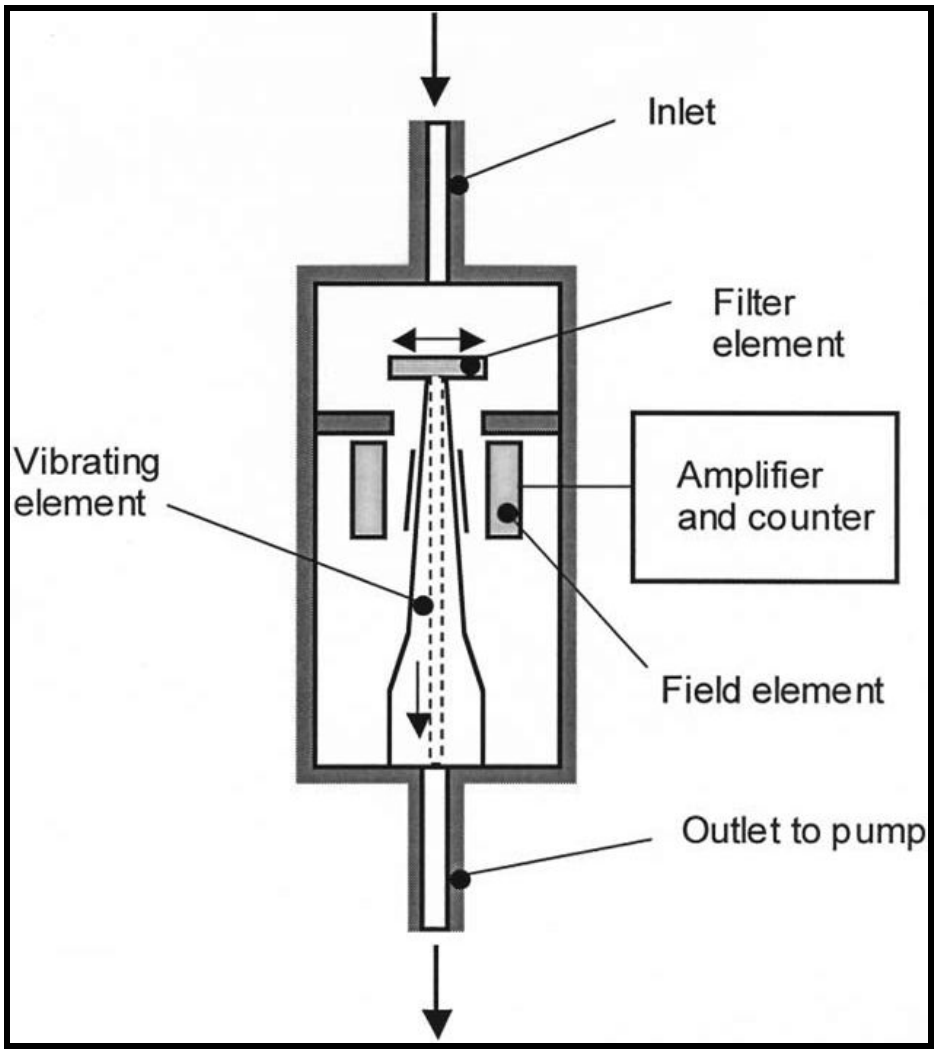



Fig 1. The basic principle of TEOMs, internal diagram

The TEOM monitor technique relies upon an exchangeable filter cartridge seated on the end of a hollow tapered tube. The broader end of the tube is fixed. As the air passes through the filter, particulate is deposited. The filtered air then passes through the tapered tube to a flow controller. The tapered tube with the filter on its end is maintained in oscillation in a clamped-free mode. The frequency of oscillation is dependent upon the physical characteristics of the tapered tube and the mass on its free end. As we have more pm2.5 particles on the top, the frequency

decreases and that decrement in frequency indicates the pm2.5 count in the Tapered Element Oscillating Microbalances.

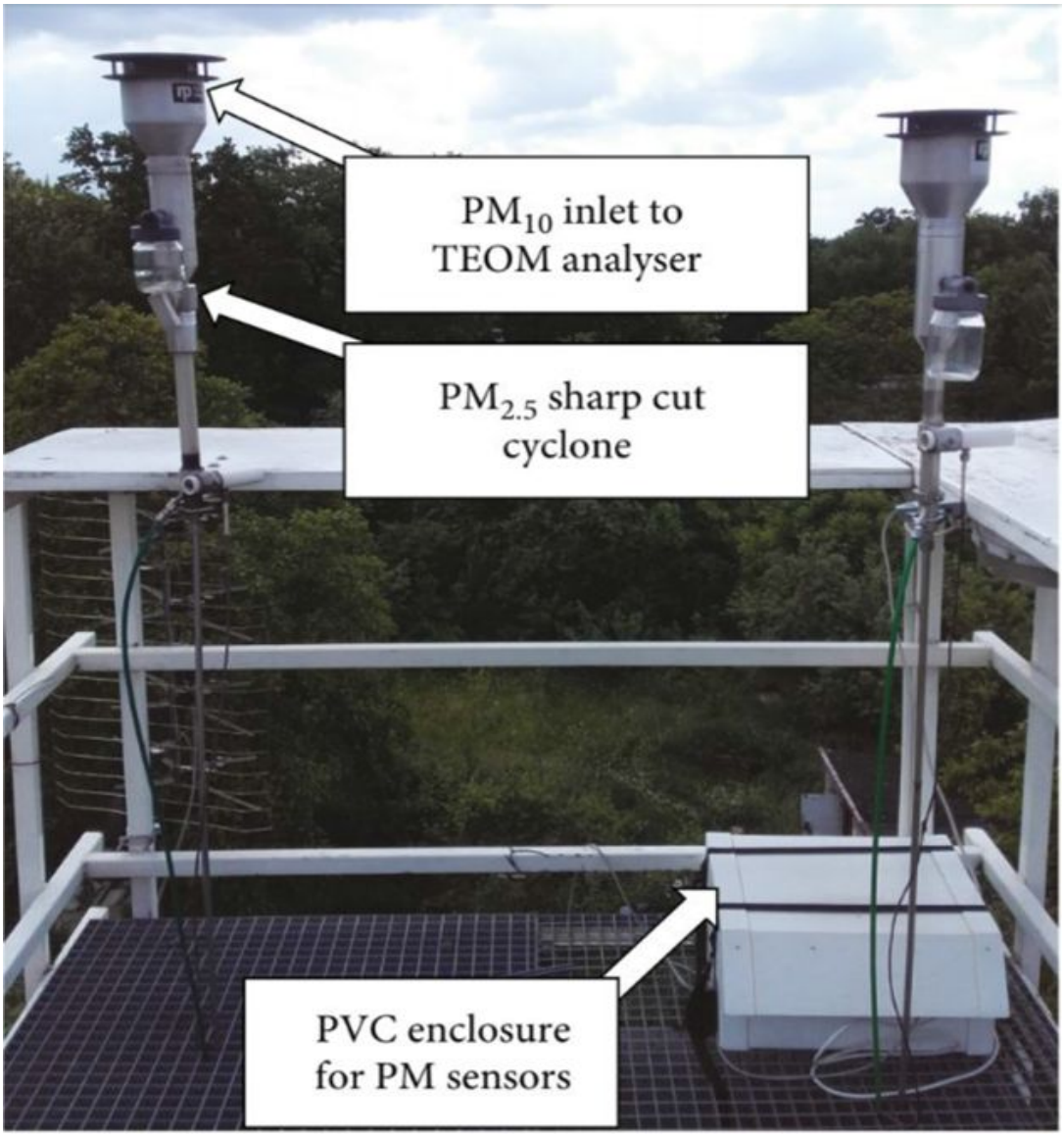



Fig 2. Outdoor deployment of TEOM sensor in government station

As it can be seen in the outdoor deployed TEOM, there are many other devices that need to be installed along with, that results in significant jumps in price even after the expensive ($20,000) single sensor.

## 3.2 E-BAMs (Environmental Beta Attenuation Monitors)

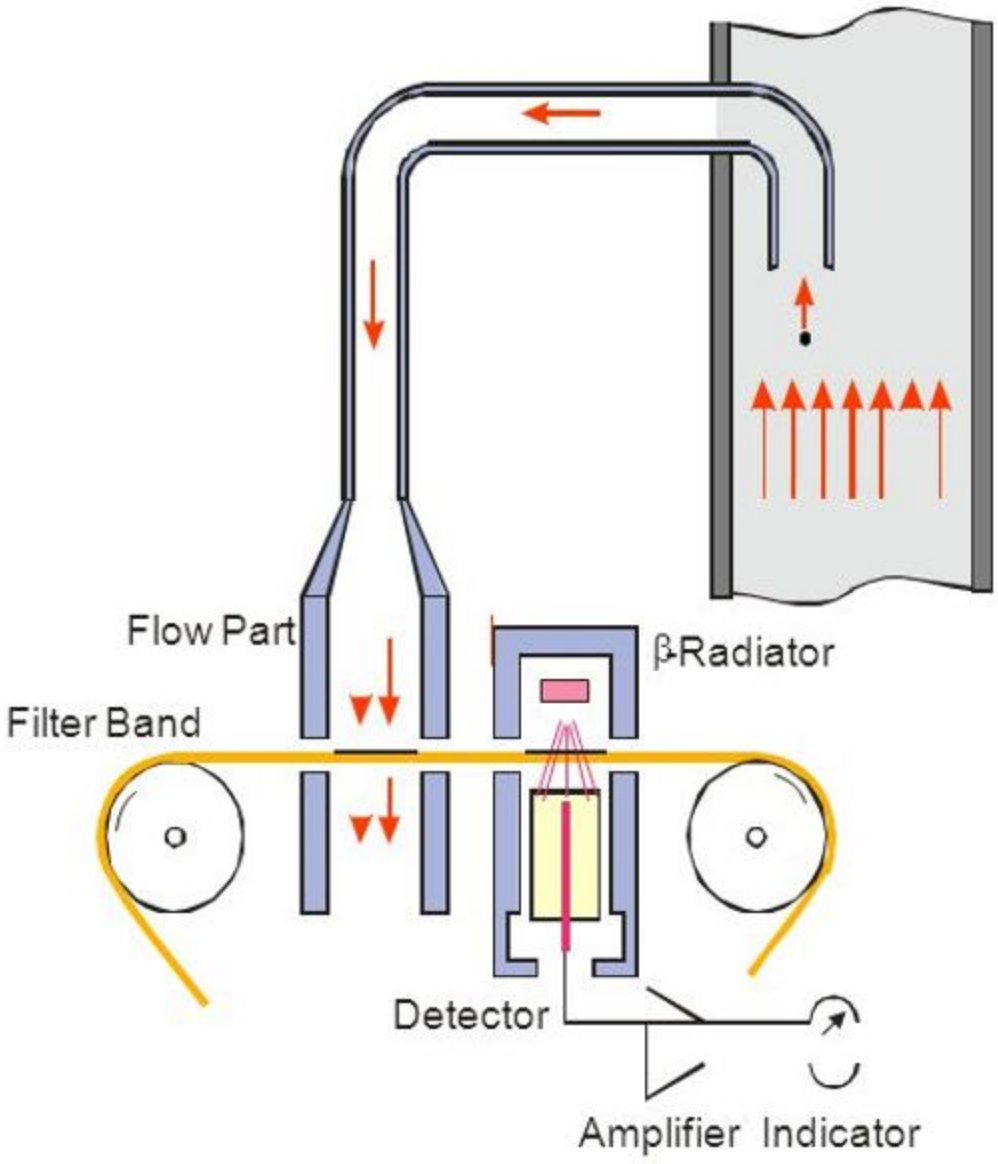



Fig 3. The basic principle involved in Beta Attenuation Monitors

The main principle is based on a kind of Bouguer (Lambert-Beer) law. Which states that the amount by which a factual matter attenuates the flow of beta radiation (electrons) is exponentially dependent on its mass and not on any other feature (such as density, chemical composition or some optical or electrical properties) of this matter.[11]-[14]

## 3.3 Optical Sensors (Low-Cost Sensors)

Currently, all market available low-cost sensors use the principle of optical scattering for pm2.5 analysis. Low-cost sensors find colossal application in Air Conditioners, Home-based Air quality monitors, Air Purifiers etc. Airflow is directed in the sensor. The PM particles scatter the light thrown on them inside the sensor.

Optical scattering runs on the principle of scattering of light on colliding with a dimensionally small particle. The scattered light is measured and converted to the electrical signal. The electrical signal indicates the amount of light scatted by the sample air. Optical Scattering based sensors are beneficial because:

1. Low current, voltage requirement
2. Small size and portability
3. Output can be obtained at a higher frequency.
4. Cheap Costs

This method carries many inaccuracies with it. The light can sometimes scatter particles other than PM2.5 also. The presence of high humidity leads to a change in the size of particles present in the air. Hence we get the wrong information regarding the PM2.5 particles present.

Light scattering-based PM sensors measure optical-equivalent PM size, which may be different from the real geometric size and thus compromises the detection accuracy. Furthermore, the ambient humidity also interferes with the optical detection of PM. Such inaccuracies make these sensors unreliable and not trustworthy to be used in government based stations.

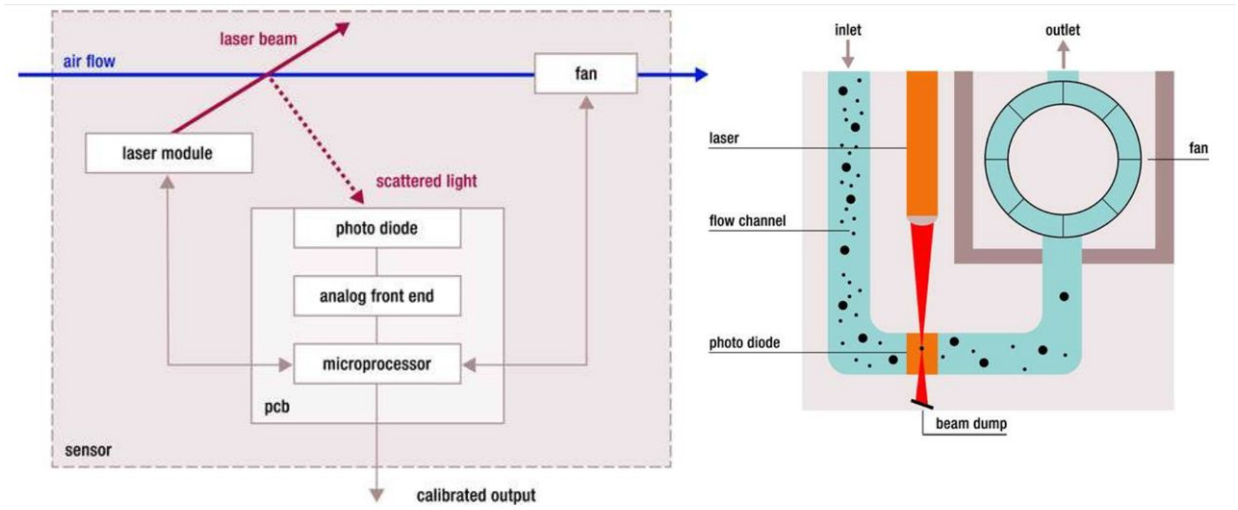



Fig 4. Flow and dynamic of Sensirion Optical based PM Sensor

## 3.4 Problems with conventional methods

The conventional sensors come with the famous accuracy vs cost trade-off. The expensive sensors cost as costly as $20,000, and low-cost sensors are around 100 dollars only but come with the low accuracy of approximately 80% averagely. The current stations/networks fail to deliver their purpose to the citizens.

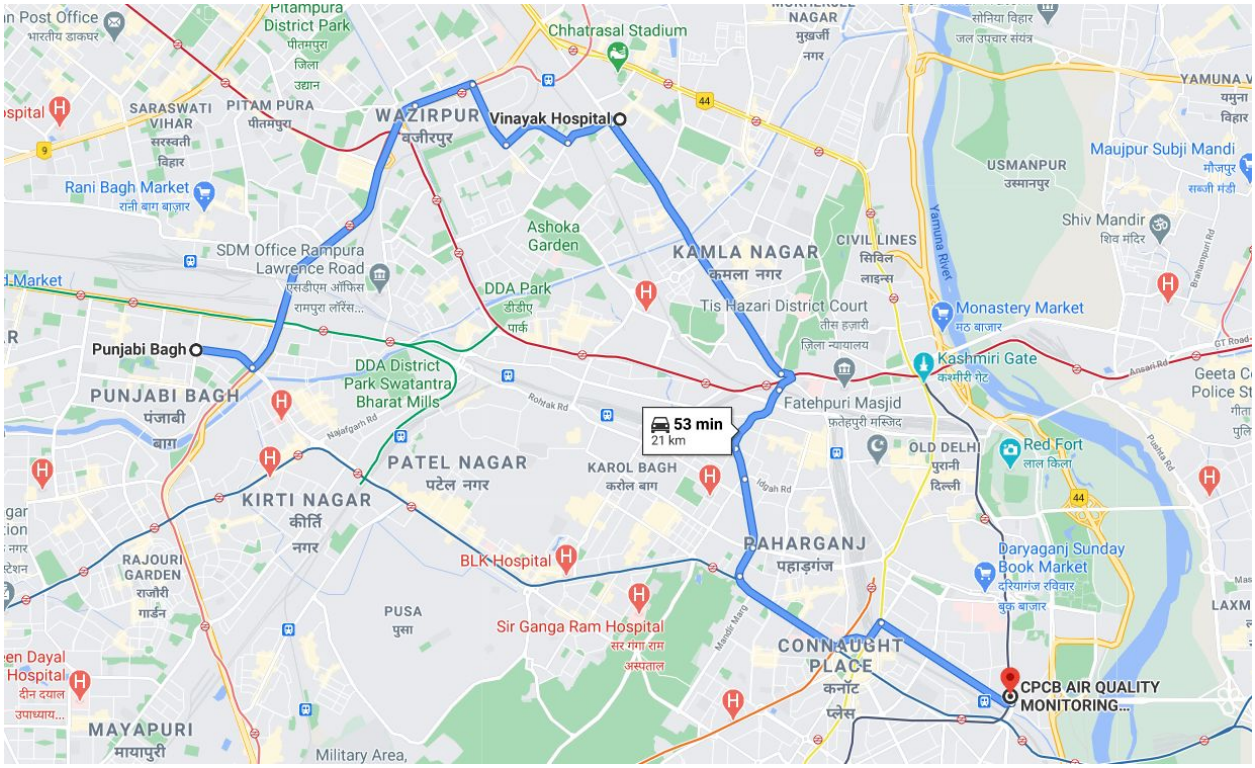



Fig 5. Air Quality station(CPCB) 12kms away from my route of 9kms from home(Punjabi Bagh) to work(Vinayak Hospital)

We can consider an example of daily travels like from home to office and from office to home. In Delhi, the average distance between the workplace and shelter is around 8kms. If someone is travelling from a particular passage daily, he would be concerned with the air quality precisely in that region only. But current sensor networks tell him the air quality roughly 5kms from his area of concern.

This can only be tackled if we could develop PM2.5 sensors at such an affordable price that enables the government to establish a relatively dense network of Air Quality Monitoring Stations. Ideally, PM monitoring should be person to person. I should be aware of the quality of air that I am inhaling.

# 4. Upcoming Technologies and Innovations

We need sensors that are:

1. Stable in their operation
2. Reliable in extreme weather conditions
3. Don't age up fast(due to dust accumulation or any reason)
4. Give good results in high humidity conditions as well.

The emerging micro-scaled technologies are helping in this field to develop sensors that are small affordable and more accurate. Various miniaturisation ideas are emerging in current times. Use of Micro Electric Mechanical Systems(MEMS) sensors appear to give good results in air monitoring. There is a need for more research in PM monitoring.

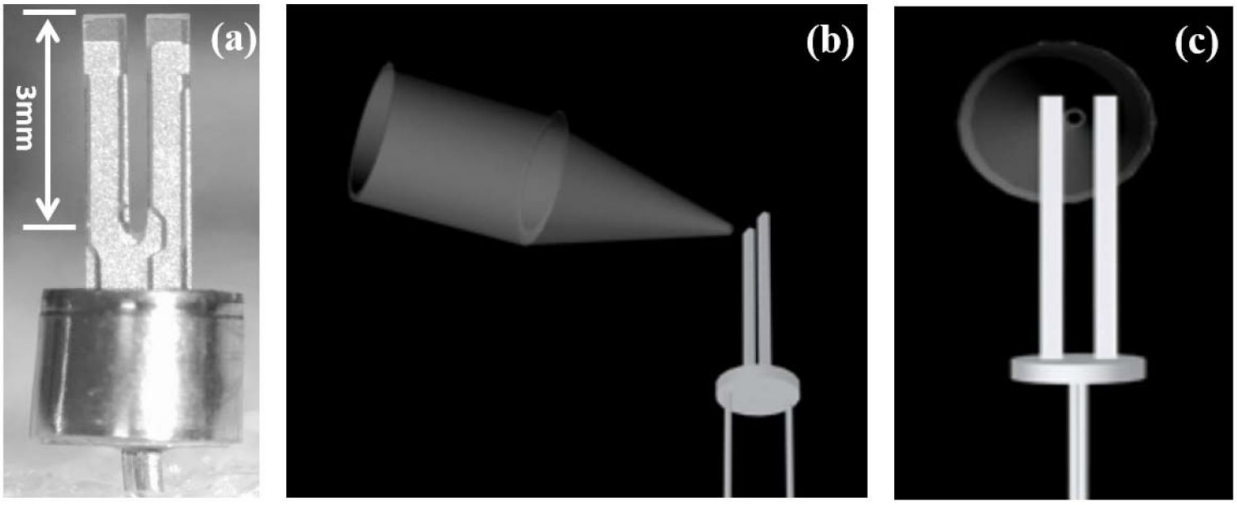



Fig 5.a Optical Image of an opened MQTF.

Fig 6 Side view and Fig 5.c front view of the 3D drawings showing the configuration of the nozzle and the MQTF in the detection chamber.

Micro Quartz Tuning Fork(MQTF) based sensing platforms are emerging in many chemical detecting and environment monitoring applications. It is attractive to develop PM sensor using MQTFs as it is intrinsically tiny, low cost(10 Cents) and highly sensitive to slight changes in masses.

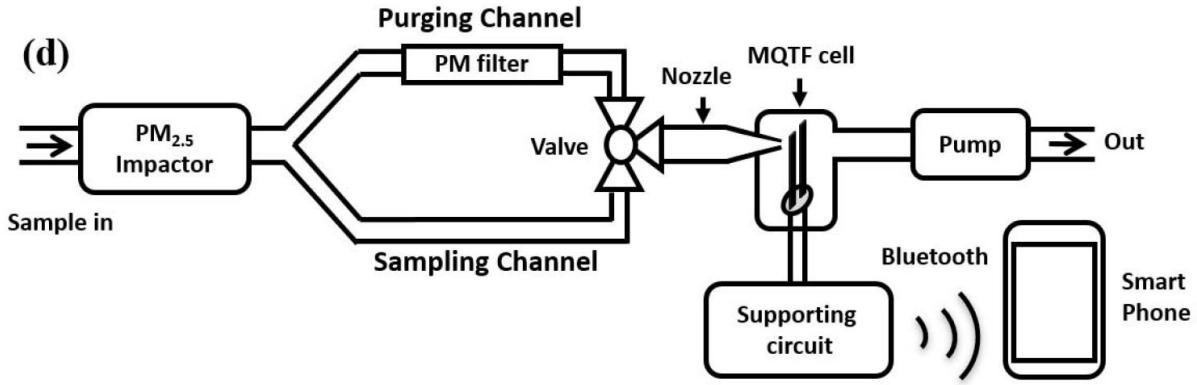



Fig 7. Schematic of the sensor based on MQTF

MQTF has a fixed tuning fork frequency, i.e. 32768 Hz. This frequency depends only on the physical parameters like mass of the tuning fork. The tuning fork is coated with a chemical adhesive that is known to stick PM2.5 particles only. The adhesive polymer is Elmer's repositionable stick with the composition of mainly poly-vinyl-pyrrolidone (PVP). When the sensor is exposed to ambient air, the PM2.5 particles are stuck to it and change in frequency can be measured. The frequency shift spatiotemporally gives out the difference in mass hence the PM deposition on MQTF.

# 5. Frequency Calculations

## 5.1 System Architecture

Our system has a variable frequency input initially at 32.768 kHz which is generated by an RTC (Real-Time Circuit) Module DS1307. This module acts as an I2C slave to the FPGA master. This variable frequency is routed to a custom frequency and time period measurement block designed in VHDL which also takes the 50 MHz frequency from the onboard oscillator. The exact frequency measured is then sent to the PM 2.5 Concentration Counter which then displays the concentration on the onboard LCD or transfers it to a smart-phone via Bluetooth.

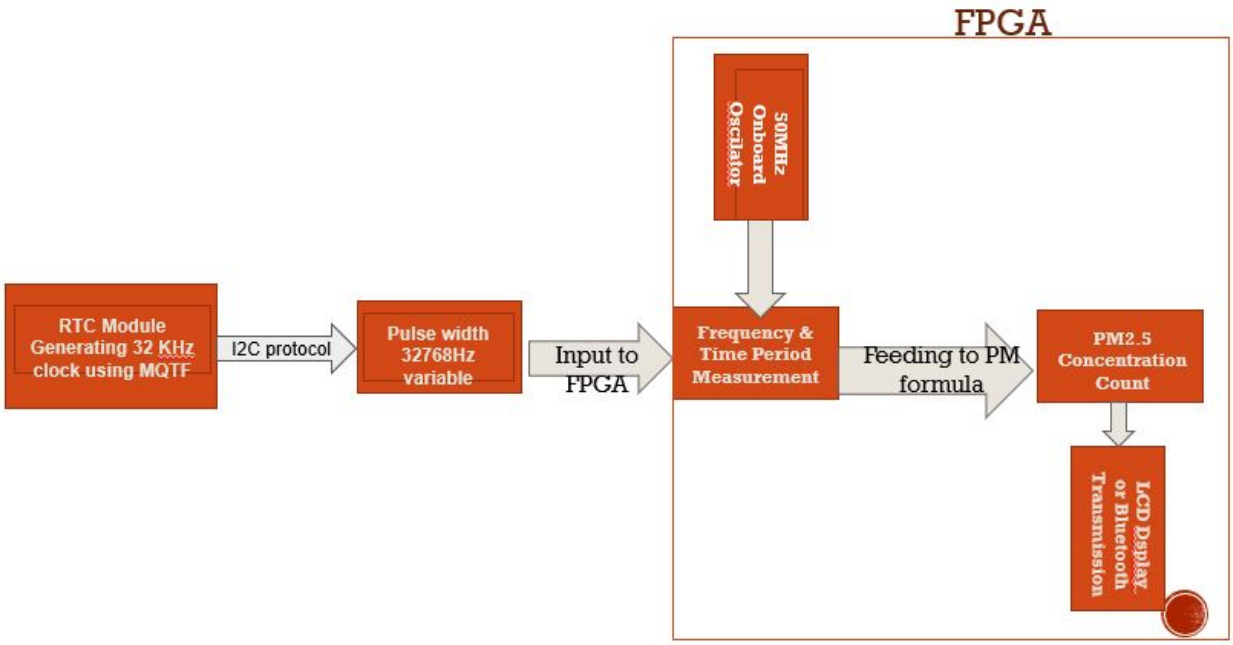



Fig 8 Frequency measurement system architecture

## 5.2 Frequency Measurement Algorithm

The method depends on measuring the period T of one clock cycle of the measured clock signal and then the frequency is simply the reciprocal of this period. One cycle period is measured by the aid of an already known internal clock which has much higher frequency than the measured clock frequency. In our design the frequency of the internal clock is equal to 50 MHz. The measured clock is used as a CLOCK ENABLE "CE". The first positive edge enables the counting process and the second positive edge disables the counting process. The counter counts the internal clock cycles and then the period T is simply the number of counted cycles multiplied by the period Ti of the internal clock.

External clock frequency = 32.768 kHz

Internal clock frequency = 50 MHz

50Mhz / 32.768 kHz ≅ $(1525)_{10}$ Thus we used an 11 bit counter for this purpose.

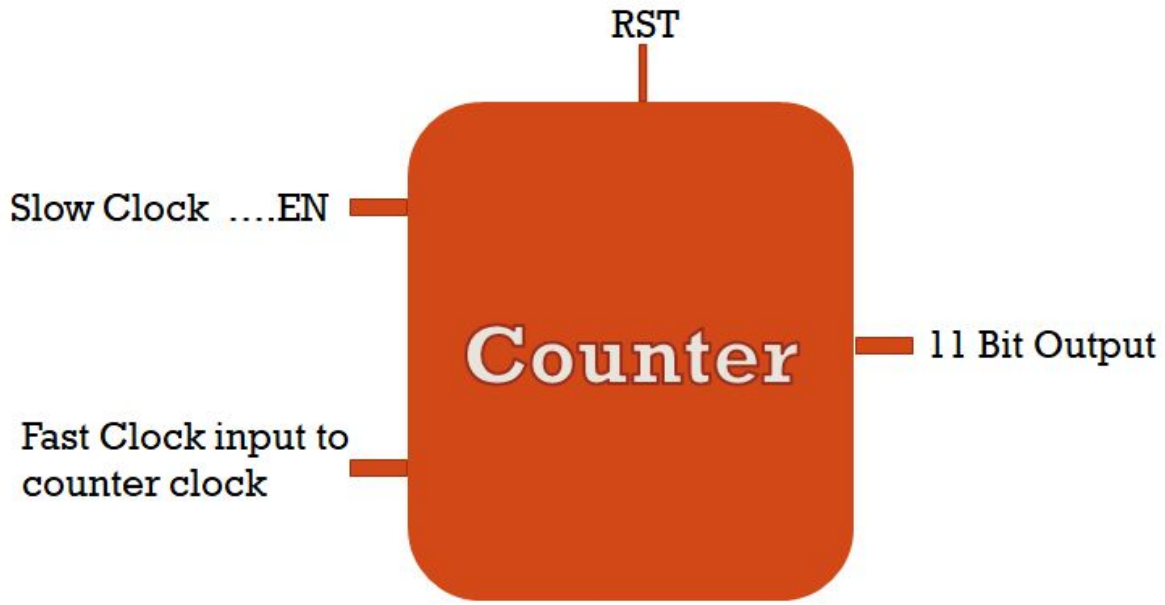



Fig. 9: 11 bit counter to count clock cycles

This method of measurement is efficient for small frequencies. The error with this measurement technique is equal to ± Ti, so the frequency of the measured clock signal will be 1/(T ± Ti). If we try to measure high frequencies in the range of the internal clock with this method then we will get a large error, because the error ±Ti represents a significant number in comparison with the period T.

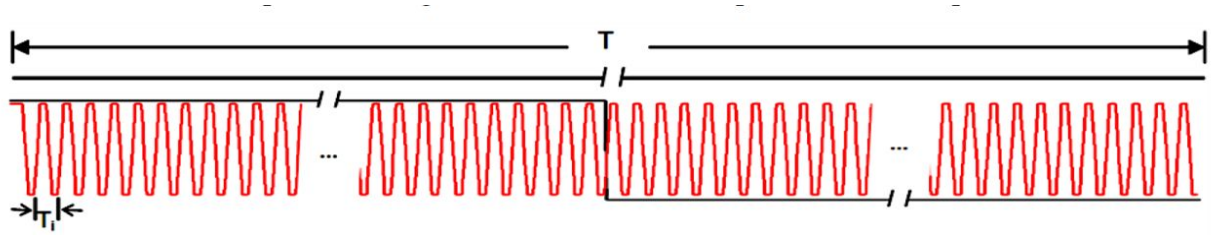



Figure 3.2: measuring the period (T) of one cycle.

## 5.3 VHDL Code

Due to the nature of arithmetic calculations involved, we included the IEEE.NUMERIC_STD library. The frequency comparison entity takes 3 inputs: fast clock, slow clock and reset and outputs the number of fast clock cycles within one slow clock cycle.

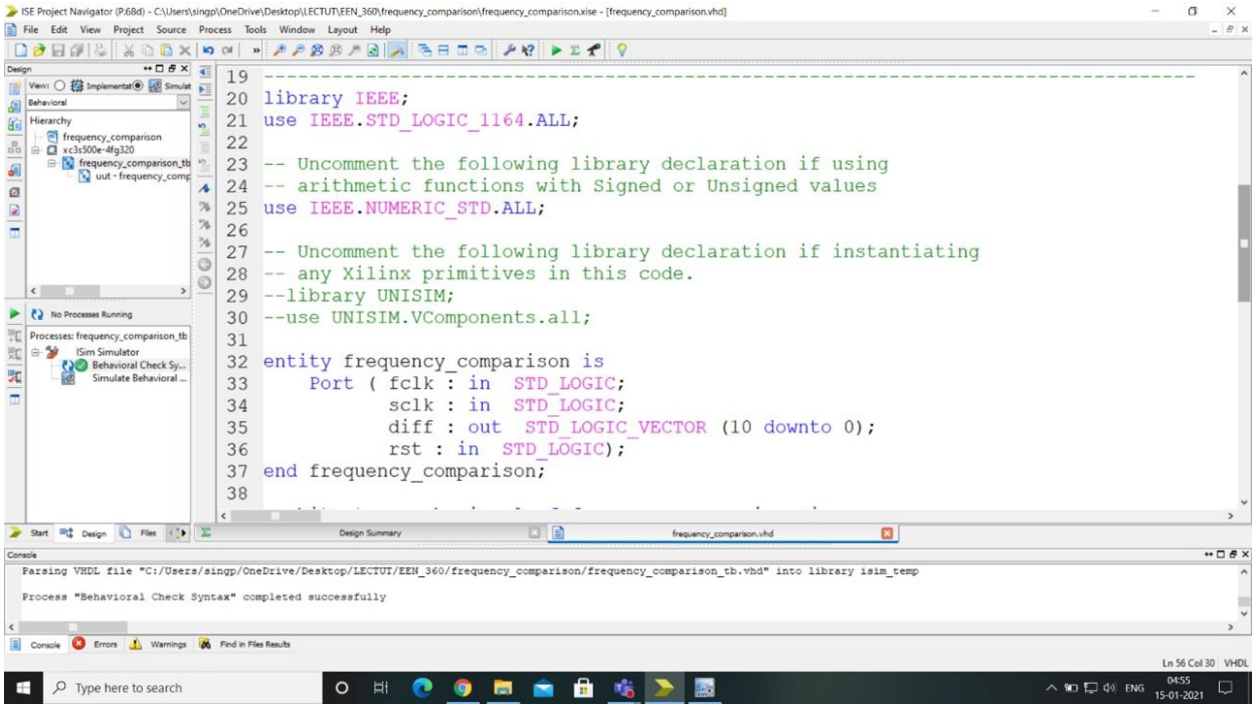



Figure 10: Frequency comparison VHDL code (1)

The reset input is connected to an onboard switch and it brings the counter to a valid state of all zeros. The *start* signal is actually the active low enable signal of the counter. Whenever a negative edge of *sclk* is encountered, the counter resets itself to clear out any garbage value. The first negative edge of *sclk* enables the counter (*start = 0*) and the subsequent edge disables it (*start = 1*). Thus the counter counts the number of *fclk* cycles in every alternate *sclk* cycle. Special attention is paid to type conversion while counting as *STD_LOGIC_VECTOR* does not allow arithmetic operations. All the arithmetic operations are done on *unsigned* type and then converted to *STD_LOGIC_VECTOR*.

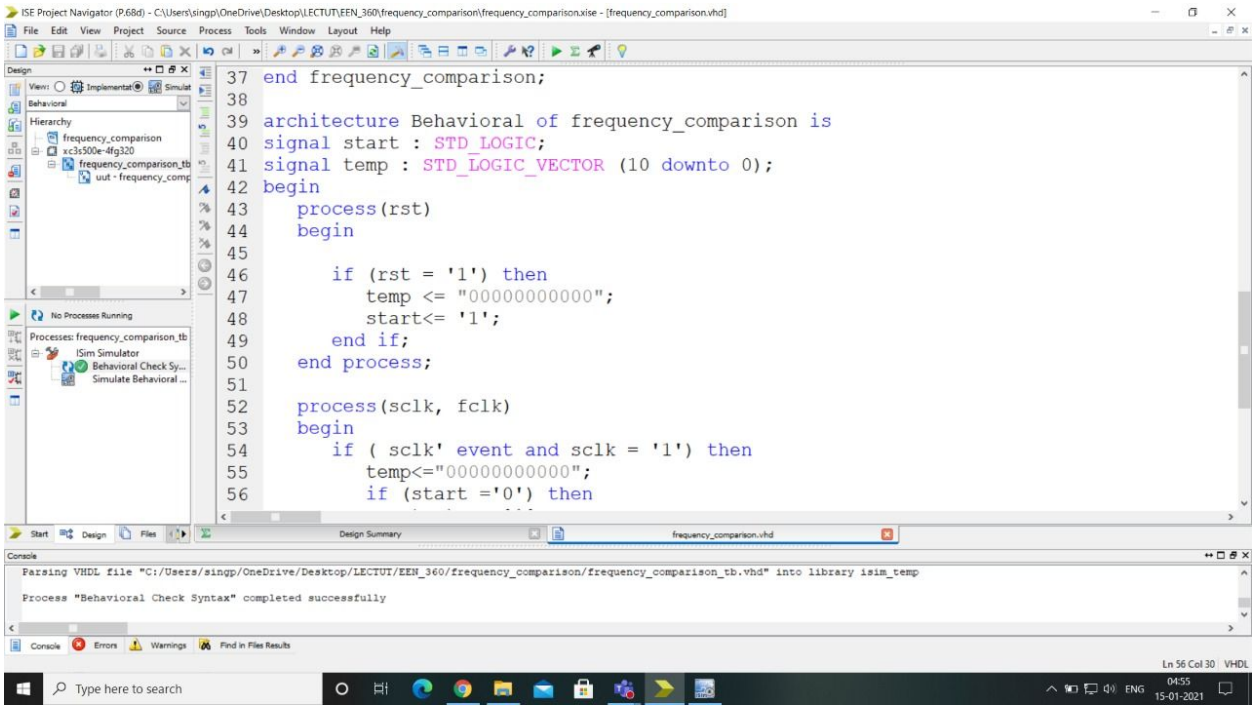



Figure 11: Frequency comparison VHDL code (2)

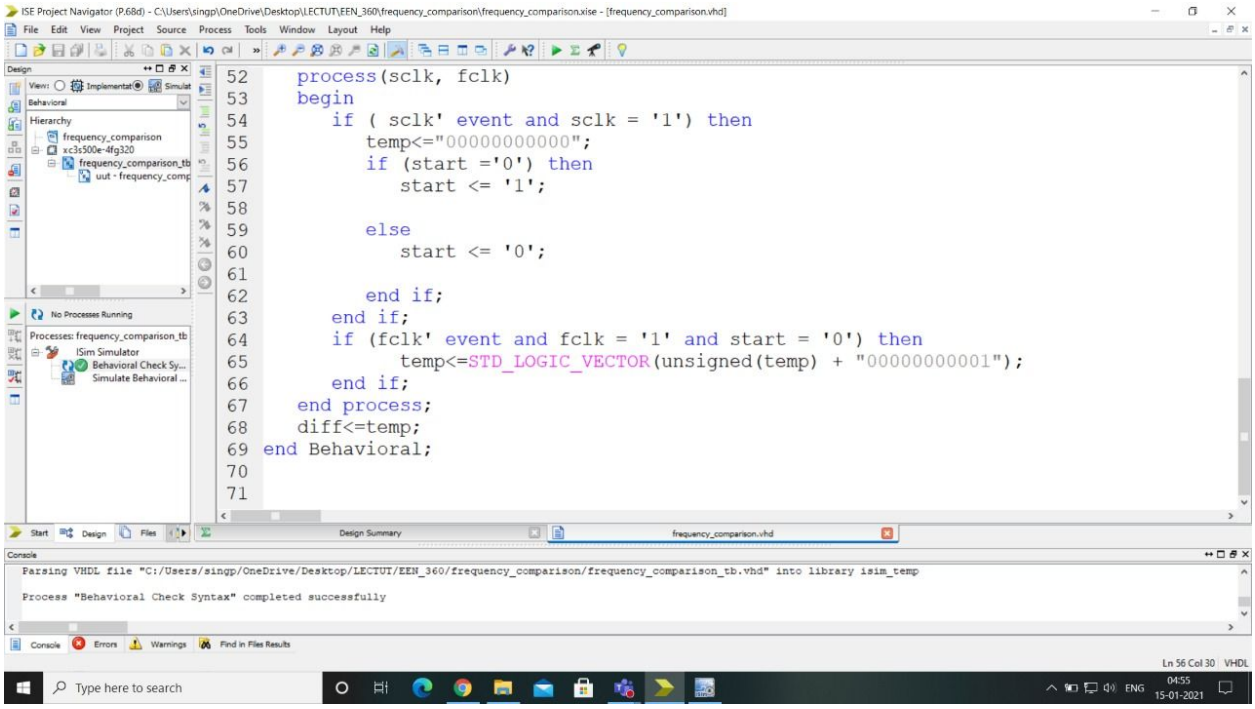



Figure 12: Frequency comparison VHDL code (3)

# 6. Interfacing with DS1307 Module

## 6.1 I2C Protocol

The I2C bus is idle when both SCL and SDA are at a logic 1 level. The master initiates a data transfer by issuing a START condition, which is a high to low transition on the SDA line while the SCL line is high. The bus is considered to be busy after the START condition. After the START condition, a slave address is sent out on the bus by the master. This address is 7 bits long followed by an eighth bit which is a data direction bit (R/W) where a 0 indicates a write from the master to the slave and a 1 indicates a read from the slave to the master. The master, who is controlling the SCL line, will send out the bits on the SDA line, one bit per clock cycle of the SCL line, with the most significant bit sent out first. The value on the SDA line can be changed only when the SCL line is at a low.


Fig. 13 (a) Start condition (b) Stop Condition

The slave device whose address matches the address that is being sent out by the master will respond with an acknowledgment bit on the SDA line by pulling the SDA line low during the ninth clock cycle of the SCL line as shown in Figure 3. The direction bit (R/W) determines whether the master or the slave will be the transmitter in the subsequent data transmission after the sending of the slave address. Every byte put on the SDA line for transmission must be 8-bits long with the most significant bit first. Except for the START and STOP conditions, the SDA line must not change when the SCL line is high. The number of bytes that can be transmitted is unrestricted. Each byte has to be followed by an acknowledge bit. The acknowledge related clock pulse is generated by the master. The transmitter releases the SDA line (sets it high) during the acknowledge clock pulse, and the receiver must pull down the SDA line during the acknowledge clock pulse to acknowledge the receipt of the byte. The one exception is when a master-receiver is involved in a transfer. In this case the master-receiver must signal the end of data to the slave-transmitter by not generating an acknowledgement on the last byte that was clocked out of the slave. To signal the end of data transfer, the master will send a STOP condition by pulling the SDA line from low to high while the SCL line is at a high. Instead of sending a STOP condition,

the master can send a repeated START condition so that the master can change the direction of the data transmission without having to release the bus.

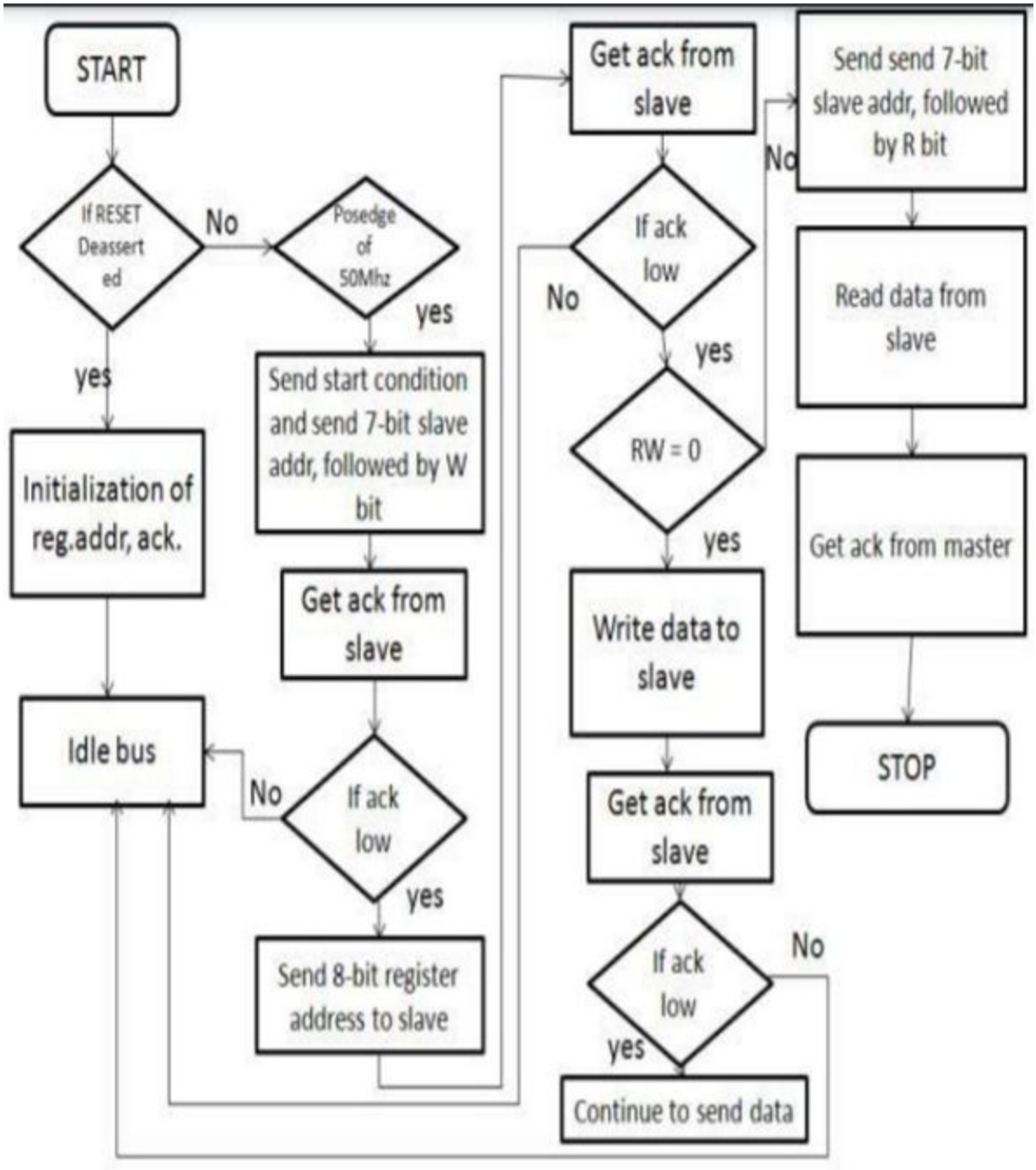



Fig. 14 Flowchart for I2C communication.

The I2C bus has two modes of operation: master transmitter and master receiver. The I 2C master bus initiates data transfer and can drive both SDA and SCL lines. Slave device (DS1307) is addressed by the master. It can issue only data on the SDA line. In master transmission mode,

after the initiation of the START sequence, the master sends out a slave address. The address byte contains the 7 bit DS1307 address, which is 1101000, followed by the direction bit (R/w ). After receiving and decoding the address byte the device outputs acknowledge on the SDA line. After the DS1307 acknowledges the slave address + write bit, the master transmits a register address to the DS1307 this will set the register pointer on the DS1307. The master will then begin transmitting each byte of data with the DS1307 acknowledging each byte received. The master will generate a stop condition to terminate the data write. Our application is to generate a square wave output of frequency 32.768kHz. This is possible by modifying the control register of DS1307 which is present at address 07H.

| BIT 7 | BIT 6 | BIT 5 | BIT 4 | BIT 3 | BIT 2 | BIT 1 | BIT 0 |
|---|---|---|---|---|---|---|---|
| OUT | 0 | 0 | SQWE | 0 | 0 | RS1 | RS0 |

Fig. 15 Control register

**Bit 7: Output Control (OUT).** This bit controls the output level of the SQW/OUT pin when the square-wave output is disabled. If SQWE = 0, the logic level on the SQW/OUT pin is 1 if OUT = 1 and is 0 if OUT = 0. On initial application of power to the device, this bit is typically set to a 0.

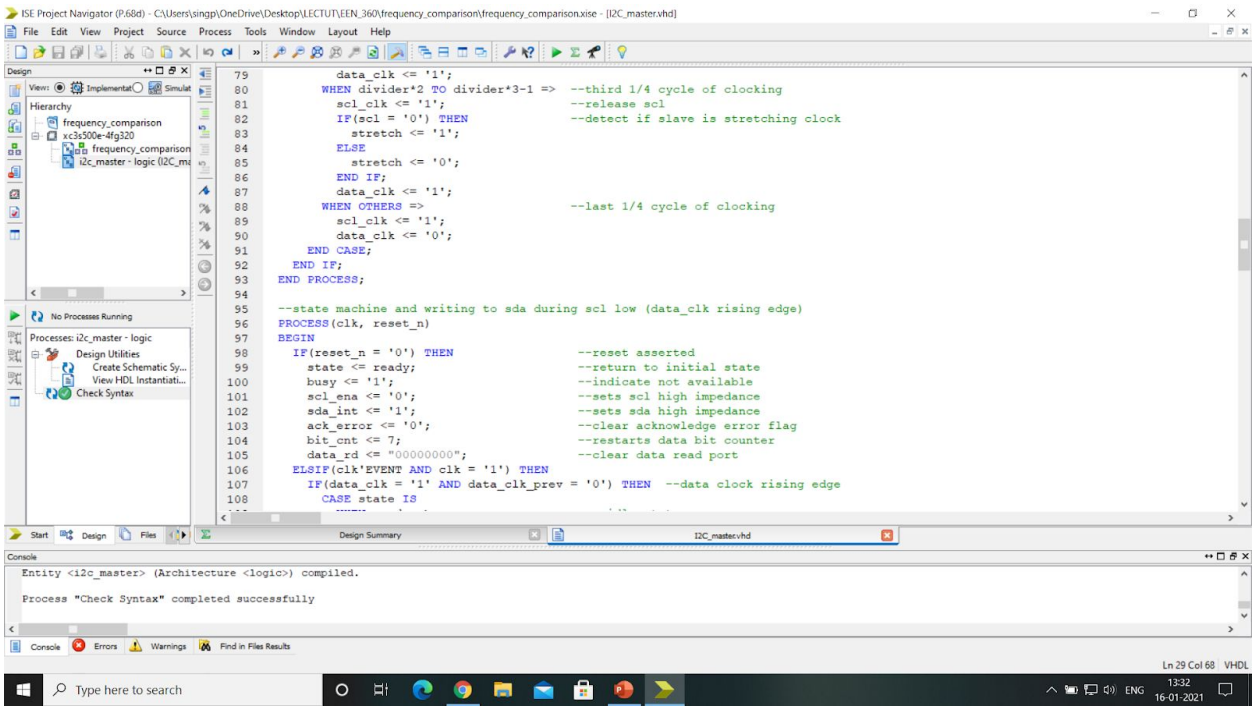
**Bit 4: Square-Wave Enable (SQWE).** This bit, when set to logic 1, enables the oscillator output. The frequency of the square-wave output depends upon the value of the RS0 and RS1 bits. With the square-wave output set to 1Hz, the clock registers update on the falling edge of the square wave. On initial application of power to the device, this bit is typically set to a 0.

**Bits 1 and 0: Rate Select (RS[1:0]).** These bits control the frequency of the square-wave output when the squarewave output has been enabled. The following table lists the square-wave frequencies that can be selected with the RS bits. On initial application of power to the device, these bits are typically set to a 1.

| RS1 | RS0 | SQW/OUT OUTPUT | SQWE | OUT |
|---|---|---|---|---|
| 0 | 0 | 1Hz | 1 | X |
| 0 | 1 | 4.096kHz | 1 | X |
| 1 | 0 | 8.192kHz | 1 | X |
| 1 | 1 | 32.768kHz | 1 | X |
| X | X | 0 | 0 | 0 |
| X | X | 1 | 0 | 1 |

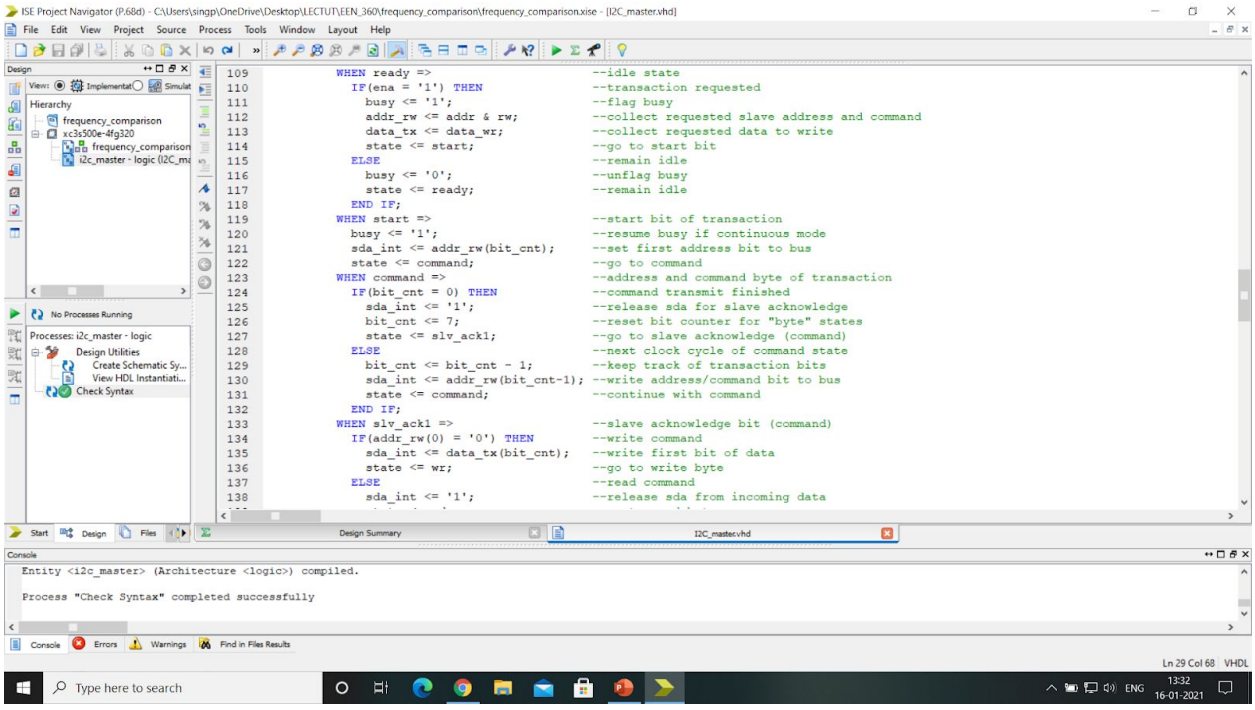
Fig. 16 Bit manipulation in control register

# 6.2 VHDL Code



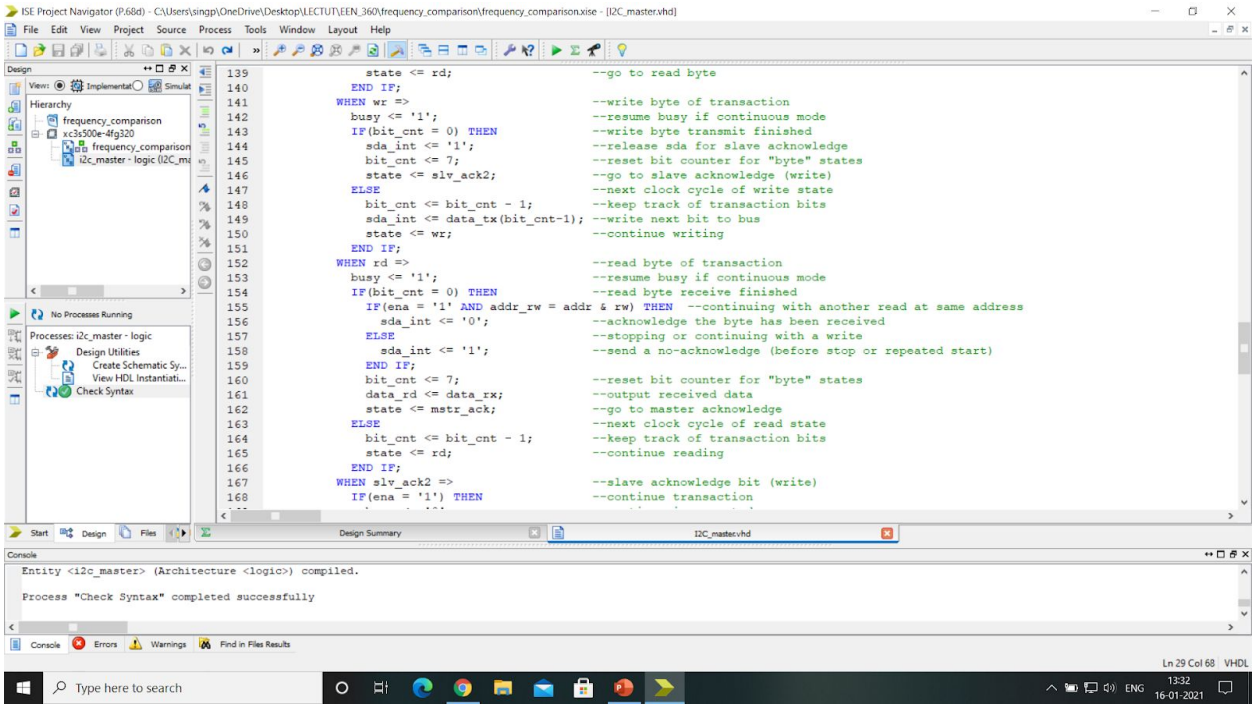
```
79            data_clk <= '1';
80          WHEN divider*2 TO divider*3-1 =>  --third 1/4 cycle of clocking
81            scl_clk <= '1';                 --release scl
82            IF(scl = '0') THEN              --detect if slave is stretching clock
83              stretch <= '1';
84            ELSE
85              stretch <= '0';
86            END IF;
87            data_clk <= '1';
88          WHEN OTHERS =>                    --last 1/4 cycle of clocking
89            scl_clk <= '1';
90            data_clk <= '0';
91        END CASE;
92      END IF;
93    END PROCESS;
94
95    --state machine and writing to sda during scl low (data_clk rising edge)
96    PROCESS(clk, reset_n)
97    BEGIN
98      IF(reset_n = '0') THEN                --reset asserted
99        state <= ready;                     --return to initial state
100       busy <= '1';                        --indicate not available
101       scl_ena <= '0';                     --sets scl high impedance
102       sda_int <= '1';                     --sets sda high impedance
103       ack_error <= '0';                   --clear acknowledge error flag
104       bit_cnt <= 7;                       --restarts data bit counter
105       data_rd <= "00000000";              --clear data read port
106     ELSIF(clk'EVENT AND clk = '1') THEN
107       IF(data_clk = '1' AND data_clk_prev = '0') THEN  --data clock rising edge
108         CASE state IS
```

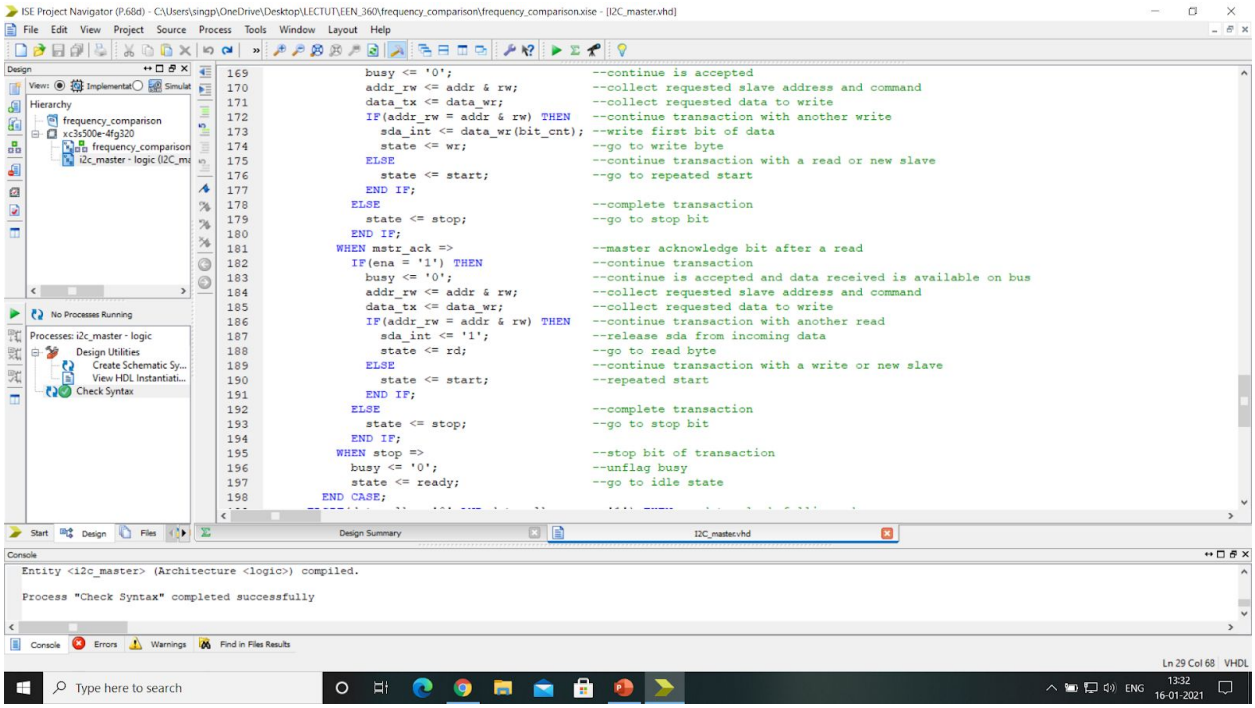
```
Entity <i2c_master> (Architecture <logic>) compiled.

Process "Check Syntax" completed successfully
```



```
109         WHEN ready =>                     --idle state
110           IF(ena = '1') THEN              --transaction requested
111             busy <= '1';                  --flag busy
112             addr_rw <= addr & rw;         --collect requested slave address and command
113             data_tx <= data_wr;           --collect requested data to write
114             state <= start;               --go to start bit
115           ELSE                            --remain idle
116             busy <= '0';                  --unflag busy
117             state <= ready;               --remain idle
118           END IF;
119         WHEN start =>                     --start bit of transaction
120           busy <= '1';                    --resume busy if continuous mode
121           sda_int <= addr_rw(bit_cnt);    --set first address bit to bus
122           state <= command;               --go to command
123         WHEN command =>                   --address and command byte of transaction
124           IF(bit_cnt = 0) THEN            --command transmit finished
125             sda_int <= '1';               --release sda for slave acknowledge
126             bit_cnt <= 7;                 --reset bit counter for "byte" states
127             state <= slv_ack1;            --go to slave acknowledge (command)
128           ELSE                            --next clock cycle of command state
129             bit_cnt <= bit_cnt - 1;       --keep track of transaction bits
130             sda_int <= addr_rw(bit_cnt-1); --write address/command bit to bus
131             state <= command;             --continue with command
132           END IF;
133         WHEN slv_ack1 =>                  --slave acknowledge bit (command)
134           IF(addr_rw(0) = '0') THEN       --write command
135             sda_int <= data_tx(bit_cnt);  --write first bit of data
136             state <= wr;                  --go to write byte
137           ELSE                            --read command
138             sda_int <= '1';               --release sda from incoming data
```

```
Entity <i2c_master> (Architecture <logic>) compiled.

Process "Check Syntax" completed successfully
```


ISE Project Navigator (P.68d) - C:\Users\singp\OneDrive\Desktop\LECTUT\EEN_360\frequency_comparison\frequency_comparison.xise - [I2C_master.vhd]
File Edit View Project Source Process Tools Window Layout Help
Design
Hierarchy
frequency_comparison
xc3s500e-4fg320
frequency_comparison
i2c_master - logic (I2C_ma
No Processes Running
Processes: i2c_master - logic
Design Utilities
Create Schematic Sy...
View HDL Instantiati...
Check Syntax
139 state <= rd; --go to read byte
140 END IF;
141 WHEN wr => --write byte of transaction
142 busy <= '1'; --resume busy if continuous mode
143 IF(bit_cnt = 0) THEN --write byte transmit finished
144 sda_int <= '1'; --release sda for slave acknowledge
145 bit_cnt <= 7; --reset bit counter for "byte" states
146 state <= slv_ack2; --go to slave acknowledge (write)
147 ELSE --next clock cycle of write state
148 bit_cnt <= bit_cnt - 1; --keep track of transaction bits
149 sda_int <= data_tx(bit_cnt-1); --write next bit to bus
150 state <= wr; --continue writing
151 END IF;
152 WHEN rd => --read byte of transaction
153 busy <= '1'; --resume busy if continuous mode
154 IF(bit_cnt = 0) THEN --read byte receive finished
155 IF(ena = '1' AND addr_rw = addr & rw) THEN --continuing with another read at same address
156 sda_int <= '0'; --acknowledge the byte has been received
157 ELSE --stopping or continuing with a write
158 sda_int <= '1'; --send a no-acknowledge (before stop or repeated start)
159 END IF;
160 bit_cnt <= 7; --reset bit counter for "byte" states
161 data_rd <= data_rx; --output received data
162 state <= mstr_ack; --go to master acknowledge
163 ELSE --next clock cycle of read state
164 bit_cnt <= bit_cnt - 1; --keep track of transaction bits
165 state <= rd; --continue reading
166 END IF;
167 WHEN slv_ack2 => --slave acknowledge bit (write)
168 IF(ena = '1') THEN --continue transaction
Design Summary
I2C_master.vhd
Console
Entity <i2c_master> (Architecture <logic>) compiled.
Process "Check Syntax" completed successfully
Console Errors Warnings Find in Files Results
Ln 29 Col 68 VHDL
Type here to search
ENG 13:32 16-01-2021


ISE Project Navigator (P.68d) - C:\Users\singp\OneDrive\Desktop\LECTUT\EEN_360\frequency_comparison\frequency_comparison.xise - [I2C_master.vhd]
File Edit View Project Source Process Tools Window Layout Help
Design
Hierarchy
frequency_comparison
xc3s500e-4fg320
frequency_comparison
i2c_master - logic (I2C_ma
No Processes Running
Processes: i2c_master - logic
Design Utilities
Create Schematic Sy...
View HDL Instantiati...
Check Syntax
169 busy <= '0'; --continue is accepted
170 addr_rw <= addr & rw; --collect requested slave address and command
171 data_tx <= data_wr; --collect requested data to write
172 IF(addr_rw = addr & rw) THEN --continue transaction with another write
173 sda_int <= data_wr(bit_cnt); --write first bit of data
174 state <= wr; --go to write byte
175 ELSE --continue transaction with a read or new slave
176 state <= start; --go to repeated start
177 END IF;
178 ELSE --complete transaction
179 state <= stop; --go to stop bit
180 END IF;
181 WHEN mstr_ack => --master acknowledge bit after a read
182 IF(ena = '1') THEN --continue transaction
183 busy <= '0'; --continue is accepted and data received is available on bus
184 addr_rw <= addr & rw; --collect requested slave address and command
185 data_tx <= data_wr; --collect requested data to write
186 IF(addr_rw = addr & rw) THEN --continue transaction with another read
187 sda_int <= '1'; --release sda from incoming data
188 state <= rd; --go to read byte
189 ELSE --continue transaction with a write or new slave
190 state <= start; --repeated start
191 END IF;
192 ELSE --complete transaction
193 state <= stop; --go to stop bit
194 END IF;
195 WHEN stop => --stop bit of transaction
196 busy <= '0'; --unflag busy
197 state <= ready; --go to idle state
198 END CASE;
Design Summary
I2C_master.vhd
Console
Entity <i2c_master> (Architecture <logic>) compiled.
Process "Check Syntax" completed successfully
Console Errors Warnings Find in Files Results
Ln 29 Col 68 VHDL
Type here to search
ENG 13:32 16-01-2021

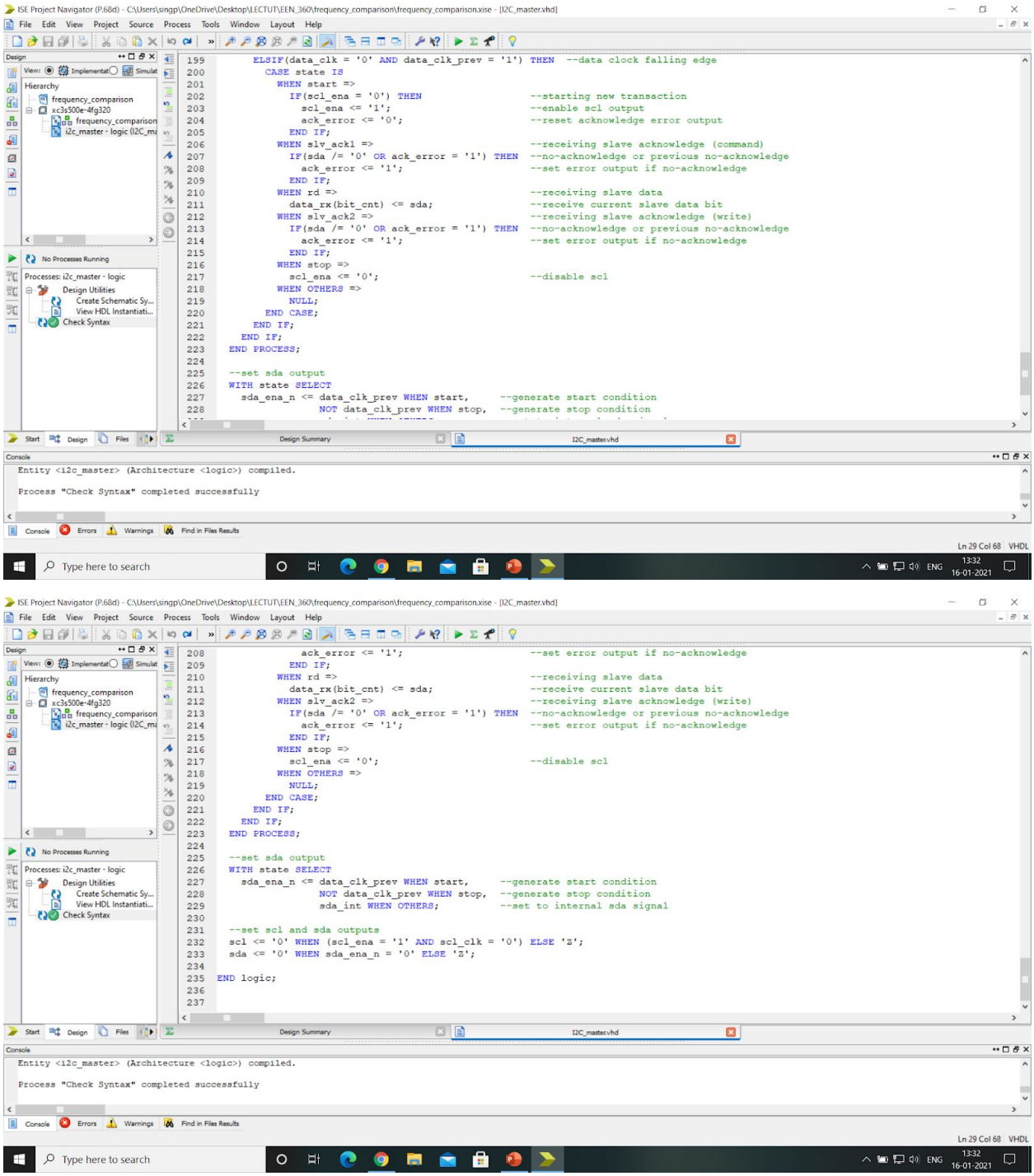






Fig. 17 I2C Protocol Implementation in VHDL

# 7. References


- “FPGA Prototyping with VHDL Examples Spartan 3E” by Pong P. Chu
- “Implementation of I2C Master Bus Protocol on FPGA” Regu Archana, Mr. J.V.Rao
- ”DS1307 64 x 8, Serial, I2C Real-Time Clock Datasheet” by maxim integrated
- FPGA Based Frequency Measurement for The Purpose of Synchronisation by Moneer Kaid Al-Bokhaiti Communications Laboratory – University of Kassel
- J. Schwartz, D. W. Dockery, and L. M. Neas, “Is daily mortality associated specifically with fine particles?” J. Air Waste Manage. Assoc., vol. 46, no. 10, pp. 927–939, Oct. 1996.
- WHO. World Health Organization Burden of Disease from the Joint Effects of Household and Ambient Air Pollution for 2012.
- K. J. Maji, M. Arora, and A. K. Dikshit, “Burden of disease attributed to ambient PM2.5 and PM10 exposure in 190 cities in China,” Environ. Sci. Pollut. Res., vol. 24, no. 12, pp. 11559–11572, Apr. 2017.
- S. Baldacci et al., “Allergy and asthma: Effects of the exposure to particulate matter and biological allergens,” Respiratory Med., vol. 109, no. 9, pp. 1089–1104, 2015.
- A. Chung et al., “Comparison of real-time instruments used to monitor airborne particulate matter,” J. Air Waste Manage. Assoc., vol. 51, no. 1, pp. 109–120, Jan. 2001.
- S. C. van der Zee, M. Strak, M. B. A. Dijkema, B. Brunekreef, and N. A. H. Janssen, “The impact of particle filtration on indoor air quality in a classroom near a highway,” Indoor Air, vol. 27, no. 2, pp. 291–302, Mar. 2017.
- G. P. Ayers, M. D. Keywood, and J. L. Gras, “TEOM vs. manual gravimetric methods for determination of PM2.5 aerosol mass concentrations,” Atmos. Environ., vol. 33, no. 22, pp. 3717–3721, 1999.
- C. S. Sloane, “Effect of composition on aerosol light scattering efficiencies,” Atmos. Environ., vol. 20, no. 5, pp. 1025–1037, 1986.
- D. W. Spencer, P. E. Biscaye, P. L. Sachs, and R. Ackerman, “Lightscattering and particulate matter results, geosecs Atlantic cruise,” Trans.- Amer. Geophys. Union, vol. 54, no. 4, p. 306, 1973.
- R. J. Charlson, N. C. Ahlquist, and H. Horvath, “On the generality of correlation of atmospheric aerosol mass concentration and light scatter,” Atmos. Environ., vol. 2, no. 5, pp. 455–464, 1968.
- F. Tsow, E. S. Forzani, and N. J. Tao, “Frequency-coded chemical sensors,” Anal. Chem., vol. 80, no. 3, pp. 606–611, Feb. 2008.
- J.-M. Friedt and É. Carry, “Introduction to the quartz tuning fork,” Amer. J. Phys., vol. 75, no. 5, pp. 415–422, 2007.